

Практическая работа 4

ASP.NET Core (.NET 10)

Тема: конечные точки, параметры маршрута, dependency injection и сервисы в middleware

Сквозное задание: «CampusRouteLab»

Параметр	Значение
Формат выполнения	Индивидуально
Тип проекта	Empty Web App / Minimal API под .NET 10
Рекомендуемое время	2 академических часа + самостоятельная доработка
Максимальный балл	100 баллов
Что сдать	Архив проекта, краткий отчёт, скриншоты тестовых запросов

Вводная

Кафедре требуется учебный диагностический веб-сервис, который одновременно показывает работу маршрутизации и встроенного механизма dependency injection. Сервис должен уметь искать учебные группы и студентов по маршрутам, выводить список зарегистрированных endpoint-ов, а также демонстрировать различия между Singleton, Scoped и Transient и корректное использование сервисов в middleware.

1. Цель работы

- Научиться создавать конечные точки minimal API методами Map()/MapGet() и связывать их с шаблонами маршрутов.
- Отработать обязательные, составные, необязательные, маршрутные параметры со значениями по умолчанию и catch-all-параметры.
- Закрепить регистрацию сервисов в IServiceCollection и показать различия между Singleton, Scoped и Transient.
- Корректно использовать сервисы внутри custom middleware и объяснять, почему scoped-зависимости нельзя помещать в конструктор middleware.

2. Постановка задачи

Необходимо разработать приложение CampusRouteLab. Это учебный веб-сервис, который хранит тестовые сведения о группах и студентах в памяти, обрабатывает запросы по заданным маршрутам и предоставляет диагностический раздел для анализа работы DI и middleware.

Приложение должно состоять из двух смысловых блоков: бизнес-маршруты (работа с группами и студентами) и диагностические маршруты (маршруты, сервисы, жизненные циклы, получение зависимостей).

3. Ограничения и технические требования

- Использовать шаблон Empty Web App и минимальную модель хостинга WebApplicationBuilder / WebApplication.
- Не использовать controllers, MVC, Razor Pages, Entity Framework Core и базу данных: все данные хранятся в памяти приложения.
- Маршруты добавлять на уровне endpoint routing. Допускается использование Map() и MapGet(), если сохраняется логика minimal API.

- Минимум половина обязательных endpoint-ов должна возвращать JSON-объекты или коллекции, а не только строки.
- Регистрацию собственных сервисов оформить через метод расширения AddCampusServices(this IServiceCollection services).
- Custom middleware вынести в отдельный класс и подключить через метод расширения UseRequestAudit().

4. Обязательные маршруты и поведение приложения

4.1. Бизнес-маршруты

Маршрут	Что должно происходить	Формат ответа
/	Краткая стартовая страница: название сервиса, назначение и список основных разделов.	Текст или JSON
/students	Вернуть список доступных учебных групп и количество студентов в каждой группе.	JSON
/students/{group}	Вернуть информацию по одной группе и список студентов этой группы. Параметр group обязателен.	JSON
/students/{group}/{id}	Вернуть одного студента по группе и идентификатору. Если студент не найден — корректный 404.	JSON / 404
/reports/{section?}	Маршрут с необязательным параметром. Если section не передан, использовать значение overview.	Текст или JSON
/portal/{module=home}/{page=index}/{id?}	Маршрут со значениями по умолчанию. В ответе явно показать, какие значения были получены системой маршрутизации.	Текст
/files/{**path}	Catch-all маршрут: вернуть весь захваченный остаток пути после /files/.	Текст

4.2. Диагностические маршруты

Маршрут	Что должно происходить	Формат ответа
/routes	Вывести все зарегистрированные конечные точки приложения через EndpointDataSource.	Текст
/diag/lifetimes	Показать идентификаторы и/или метки времени для singleton, scoped и transient в одном запросе.	JSON
/diag/lifetimes/check	Внутри одного запроса разрешить сервисы повторно и продемонстрировать, какие значения совпадают, а какие отличаются.	JSON
/diag/request-services	Получить зависимость через HttpContext.RequestServices.GetRequiredService(...) и вывести результат.	Текст или JSON
/diag/app-services	Получить singleton-сервис через app.Services.GetRequiredService(...) и вывести данные о приложении.	Текст или JSON

Важно

Имена параметров маршрута должны совпадать с именами параметров обработчика. Минимум один обработчик маршрута вынесите в отдельный метод, а не оставляйте все endpoint-ы только в лямбда-выражениях.

5. Сервисы и жизненный цикл зависимостей

В приложении необходимо зарегистрировать и использовать как минимум следующие сервисы:

Сервис	Жизненный цикл	Назначение
IStudentCatalogService / StudentCatalogService	Singleton	Хранение и выдача тестовых данных о группах и студентах.
IAppInfoService / AppInfoService	Singleton	Идентификатор экземпляра приложения и время его запуска.
IRequestContextService / RequestContextService	Scoped	Идентификатор запроса и момент создания объекта на один HTTP-запрос.
ITransientMarkerService / TransientMarkerService	Transient	Новая метка/Guid при каждом разрешении зависимости.
DiagnosticsReportService	Transient или Scoped	Композитный сервис, который собирает диагностический отчёт, получая другие сервисы через конструктор.

Требования к демонстрации жизненного цикла:

- В endpoint-е /diag/lifetimes вывести значения singleton, scoped и transient, полученные напрямую из обработчика.
- В том же ответе вывести значения, полученные из DiagnosticsReportService, чтобы было видно, какие зависимости совпадают в рамках одного запроса.
- В endpoint-е /diag/lifetimes/check показать как минимум два разрешения transient-сервиса внутри одного запроса и явно зафиксировать, что значения различаются.
- В отчёте кратко пояснить разницу между Singleton, Scoped и Transient на основе результатов тестирования.

6. Обязательные способы получения зависимостей

Способ	Где должен быть показан в работе
Через конструктор класса	В обычном сервисе, например в DiagnosticsReportService, который получает зависимости через конструктор.
Через параметр обработчика маршрута	В endpoint-ax /students/{group}, /students/{group}/{id} и /diag/lifetimes.
Через параметр InvokeAsync middleware	В custom middleware для scoped/transient-сервисов.
Через HttpContext.RequestServices	В endpoint-е /diag/request-services либо в отдельной диагностической ветке.
Через app.Services	Только для singleton-сервиса в /diag/app-services. Использовать именно GetRequiredService(...).

Методический акцент

Если зависимость обязательна для работы диагностического endpoint-а или middleware, предпочтительно использовать `GetRequiredService(...)`. `GetService(...)` допускается только при отдельной обработке ситуации, когда регистрация сервиса отсутствует.

7. Требования к custom middleware

- Создать класс `RequestAuditMiddleware` в отдельном файле и подключить его через метод расширения `UseRequestAudit()`.
- Через конструктор middleware получить `RequestDelegate` и singleton-сервис `IAppInfoService`.
- Через параметр метода `InvokeAsync` получить `IRequestContextService` и `ITransientMarkerService`.
- До вызова `next()` зафиксировать начало обработки запроса, после `next()` добавить в ответ диагностические заголовки `X-App-Instance`, `X-Request-Id` и `X-Transient-Id`.
- Для запросов в раздел `/diag` дополнительно вывести в консоль либо в ответ краткую информацию о способе разрешения зависимостей.

Критически важное требование

Scoped-сервис нельзя внедрять в конструктор middleware. Middleware создаётся один раз на всё время жизни приложения, поэтому scoped-зависимость в конструкторе приводит к ошибке времени выполнения. Если нужен scoped-сервис, его необходимо получать через `Invoke/InvokeAsync` или через `RequestServices` в рамках конкретного запроса.

8. Рекомендуемая структура проекта

```
CampusRouteLab/  
  Program.cs  
  Models/  
    Student.cs  
  Middleware/  
    RequestAuditMiddleware.cs  
    RequestAuditExtensions.cs  
  Services/  
    IStudentCatalogService.cs  
    StudentCatalogService.cs  
    IAppInfoService.cs  
    AppInfoService.cs  
    IRequestContextService.cs  
    RequestContextService.cs  
    ITransientMarkerService.cs  
    TransientMarkerService.cs  
    DiagnosticsReportService.cs  
    ServiceCollectionExtensions.cs
```

Минимальный набор сущностей можно расширять, но логика разделения на маршруты, middleware и сервисы должна быть сохранена.

Пример сигнатур, которые можно взять за основу:

```
public interface IAppInfoService  
{  
    Guid AppInstanceId { get; }  
    DateTime StartedAt { get; }  
}
```

```

public interface IRequestContextService
{
    Guid RequestId { get; }
    DateTime CreatedAt { get; }
}

public class RequestAuditMiddleware
{
    public RequestAuditMiddleware(RequestDelegate next, IAppInfoService appInfo) { ... }
    public Task InvokeAsync(HttpContext context, IRequestContextService requestContext,
        ITransientMarkerService transientMarker) { ... }
}

public static class ServiceCollectionExtensions
{
    public static IServiceCollection AddCampusServices(this IServiceCollection services) { ... }
}

```

9. Что необходимо проверить перед сдачей

- Маршрут /students/{group} корректно использует обязательный параметр group.
- Маршрут /students/{group}/{id} отдаёт данные существующего студента и корректно обрабатывает отсутствие записи.
- Маршрут /reports/{section?} работает как с параметром, так и без него.
- Маршрут /portal/{module=home}/{page=index}/{id?} показывает применение значений по умолчанию.
- Маршрут /files/{**path} захватывает произвольный остаток пути.
- Маршрут /routes действительно выводит список зарегистрированных endpoint-ов.
- Маршруты /diag/lifetimes и /diag/lifetimes/check наглядно демонстрируют различия в жизненном цикле сервисов.
- В ответах видны заголовки, добавленные custom middleware.

10. Что сдаёт студент

- Архив исходного проекта, который собирается и запускается без дополнительной ручной настройки.
- Краткий отчёт (2–3 страницы): цель, структура проекта, таблица сервисов и их жизненного цикла, результаты тестирования.
- Не менее 6 скриншотов запросов, среди которых обязательно должны быть: /routes, один маршрут с обязательным параметром, один маршрут с необязательным параметром, один маршрут со значениями по умолчанию, /diag/lifetimes и пример ответа с заголовками middleware.
- Короткое пояснение (3–5 предложений), почему scored-сервис нельзя внедрять в конструктор middleware.

11. Разбалловка

Блок оценки	Что проверяется	Баллы
Подготовка проекта	Корректный шаблон проекта, рабочий запуск, аккуратная структура каталогов, in-мемори данные.	10
Маршрутизация и	Наличие обязательных бизнес-маршрутов, корректная	20

Блок оценки	Что проверяется	Баллы
endpoint-ы	работа Map()/MapGet(), один обработчик вынесен в отдельный метод.	
Параметры маршрута	Обязательный параметр, несколько параметров, необязательный параметр, значения по умолчанию, catch-all маршрут.	20
Регистрация и получение сервисов	AddCampusServices(), конструкторная инъекция, параметр обработчика, RequestServices, app.Services.	20
Жизненный цикл сервисов	Корректное использование Singleton/Scoped/Transient и наглядная демонстрация различий в диагностических endpoint-ах.	15
Custom middleware	Отдельный класс, метод расширения, заголовки/логирование, правильное внедрение scoped-зависимости через InvokeAsync.	10
Отчёт и защита	Скриншоты, пояснения, аккуратность вывода, способность объяснить принятые решения.	5

Штрафы

Минус 5 баллов — если вместо endpoint routing использованы controllers без отдельного согласования. Минус 5 баллов — если scoped-сервис внедрён в конструктор middleware. Минус 5 баллов — если не реализован хотя бы один из обязательных видов маршрутных параметров.

12. Дополнительные баллы (по решению преподавателя)

- До +3 баллов — за аккуратное оформление ответов в TypedResults / Results с корректными статус-кодами.
- До +3 баллов — за выделение endpoint-ов в отдельные методы или extension-классы без ухудшения читаемости Program.cs.
- До +4 баллов — за удобный диагностический вывод (например, компактная таблица маршрутов или структурированный JSON-отчёт).

13. Контрольные вопросы для защиты

- Чем отличаются Singleton, Scoped и Transient в ASP.NET Core?
- Почему middleware считается long-lived объектом приложения?
- Когда уместно использовать HttpContext.RequestServices, а когда лучше обойтись явной инъекцией?
- Как связаны имена параметров маршрута и параметры обработчика endpoint-а?
- Чем обычный параметр маршрута отличается от необязательного и catch-all-параметра?

Итог

В качественной работе CampusRouteLab преподаватель должен увидеть не набор случайных маршрутов, а связный учебный проект, в котором маршрутизация, DI, жизненный цикл сервисов и custom middleware работают совместно и осмысленно.